Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	README later
Project / Solution Name	Halal Supplier Control
Name (Leader)	Hu Jay
Email & Phone (Leader)	hu.jay.hu.jay@gmail.com & +6012-942 8971
University Affiliation(s)	Xiamen University Malaysia
Live Prototype URL	https://halal-supplier-control--dedoh.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/deettoh/halal-supplier-control

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

Halal Supplier Control runs as one Replit Autoscale deployment and one RunPod Serverless endpoint. A FastAPI backend serves the React frontend and a JSON API under /api. It reads and writes a Replit PostgreSQL database holding the materials, the shared supplier certificate registry, deliveries with their label photos, supervisor decisions and the activity log. When receiving staff upload a label photo, the backend resizes it and sends it to the RunPod endpoint, which runs Qwen3-VL-8B-Instruct on vLLM behind an OpenAI-compatible API. A JSON schema fixes the fields the model returns. Five Python rules then compare the extracted brand and manufacturer with the masterlist and the supplier's certificate and return the verdict. A Covered delivery is accepted automatically, and every other verdict waits for the Halal Supervisor's decision. Only the extraction call leaves Replit, and the endpoint URL and API key are kept in Replit Secrets. If that call fails, the backend serves a stored reading for the prepared demo photos and saves any other photo for supervisor review. The system block diagram is in Appendix A.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.11, JavaScript	Backend and frontend languages	PSF-2.0; no license (ECMA-262 standard)
FastAPI 0.157.0, Uvicorn 0.54.0	Web framework and ASGI server for the API and the built frontend	MIT; BSD-3-Clause
SQLAlchemy 2.0.55, psycopg 3.3.6	Database models and the PostgreSQL driver	MIT; LGPL-3.0
Pydantic 2.14.1, python-multipart 0.0.32	Request and response validation, photo upload parsing	MIT; Apache-2.0
openai 3.35.1 (Python SDK)	Client for the RunPod endpoint's OpenAI-compatible API	Apache-2.0

Name & Version	Purpose in Project	License
rapidfuzz 3.14.6	Fuzzy brand and manufacturer matching in rule R2	MIT
Pillow 13.0.0	Resizing label photos and storing them as JPEG	MIT-CMU
React 19.3.1, Vite 8.3.5, Tailwind CSS 4.3.3	Frontend UI, build tool and styling	MIT
TanStack Query 5.104.5, wouter 3.13.0	Data fetching and routing in the frontend	MIT; Unlicense
React-aria-components 1.22.1, lucide-react 1.61.0	Accessible form controls and icons	Apache 2.0; ISC
Rambla font (@fontsource/rambla 5.3.0)	Interface typeface	OFL-1.1
vLLM 0.31.0 (RunPod worker 1.12.0)	Model server on the RunPod endpoint	Apache-2.0

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
RunPod Serverless endpoint (OpenAI-compatible chat completions API served by vLLM), RunPod	Reads each delivery label photo into structured fields	Sponsor-provided (RunPod credit)

No other external API is called. Fonts and icons are bundled into the build.

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit Autoscale deployment, Replit	Hosts the FastAPI backend and the built React frontend at the live URL	Replit Core; Autoscale, 2 vCPU / 4GiB RAM; Asia
Replit PostgreSQL, Replit	Stores materials, certificates, deliveries with photos, decisions and the activity log	Replit Core (production database of the deployment); Asia
Replit Secrets, Replit	Holds DATABASE_URL, the RunPod endpoint URL and the API key	Replit Core
RunPod Serverless, RunPod	GPU inference for the label-reading model	Pay per use from sponsor credit; 24 GB GPU tier; worker in US-TX-3
GitHub, GitHub	Source repository, pull requests and CI (GitHub Actions)	Free

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Qwen3-VL-8B-Instruct (bf16)	Qwen team (Alibaba Cloud), via Hugging Face Qwen/Qwen3-VL-8B-Instruct; served on our RunPod endpoint	Pre-trained, used as released with no fine-tuning	Apache-2.0

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Synthetic label evaluation set synthetic-labels-2026-10-22-v1 (8 photos with expected answers)	tests/eval/ in the repo; images generated with OpenAI's image_gen tool	Synthetic (AI-generated, team-created)	Apache-2.0 with the repo	N
Demo seed data: 2 fictional SMEs, 2 suppliers, a fictional certification body, materials and certificates	src/seed/seed.json in the repo	Synthetic (team-created)	Apache-2.0 with the repo	N
Demo label photos and certificate images	src/frontend/public/media/ and src/seed/certificates/ in the repo; generated with OpenAI's image_gen tool	Synthetic (AI-generated, team-created)	Apache-2.0 with the repo	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude Code (Anthropic Claude models)	Backend services, model integration, tests, documentation, pitch deck design, pitch video script and editing	Major
OpenAI Codex	Frontend build and redesign, evaluation dataset preparation, scoring script	Major
ChatGPT image generation (OpenAI image_gen)	Synthetic label photos, demo photos and certificate images	Moderate
Replit AI assistant	Frontend scaffolding for the first UI prototype	Minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Every change reached main through a pull request that needed another team member's approval, and CI ran the backend tests, lint and strict type checks plus the frontend tests, lint and build on each one. The member who opened a pull request tested the AI-assisted code in it and answered for it in review. A team member checked every generated evaluation image and its expected answers before use. Before submission, the full demo journey ran end to end on the deployed app with the live model, and the eight-label evaluation was scored against the expected answers.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Replit Core (platform, deployment, database)	Hosts the whole app as an Autoscale deployment, with Replit PostgreSQL and Replit Secrets
RunPod	US$25 RunPod credit (GPU compute)	Runs the Qwen3-VL-8B-Instruct label-reading model on a Serverless endpoint

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Both are essential to the prototype as built. Replit Autoscale hosts the whole app: FastAPI serves the React screens and the API, Replit PostgreSQL stores the supplier records, certificates, delivery photos, decisions and activity log, and Replit Secrets hold the RunPod key. RunPod Serverless runs Qwen3-VL-8B-Instruct on a 24 GB GPU for the only AI step, reading each label photo. Neither is irreplaceable: the app reaches the model through an OpenAI-compatible base URL and the database through DATABASE_URL, so either can move by changing configuration.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

No user data is collected. Every supplier, SME, certificate and label photo is synthetic, and login is simulated by a role and organisation switcher. The app stores uploaded label photos as JPEG (up to 10 MiB per upload), the fields read from them, verdicts, supervisor decisions with their reasons, and an activity log of role, action and time in the Replit PostgreSQL database. Each photo is sent over HTTPS to our own RunPod endpoint running an open-weight model, so documents don't reach a third-party LLM vendor. Every request is checked against the caller's role and organisation, and keys are held in Replit Secrets. Demo data is kept until the demo reset restores the seed. In a production deployment, Malaysia's PDPA principles would apply to staff accounts and supplier contacts through purpose notices, a retention period for label photos, encryption at rest, and access and correction requests.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

On eight AI-generated test labels, verdicts were 8 of 8 correct and fields 27 of 27, but the model guessed one of five unreadable fields instead of leaving it blank, and synthetic photos are cleaner than photos of delivered goods. Rules decide the verdict, and a label with an unread field is queued for the supervisor as reading incomplete. The system reports document coverage and doesn't verify certificate authenticity or give a halal ruling. Brand matching uses a fuzzy threshold of 90, so a lookalike brand could pass. Extraction depends on one RunPod endpoint: the first call after idle took about three minutes in two runs, and calls time out after 180 seconds. Mitigations are one active worker during the demo, stored readings for the demo photos, and a client that switches provider by base URL. Login is simulated with request headers and needs Orbyte's authentication in production.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

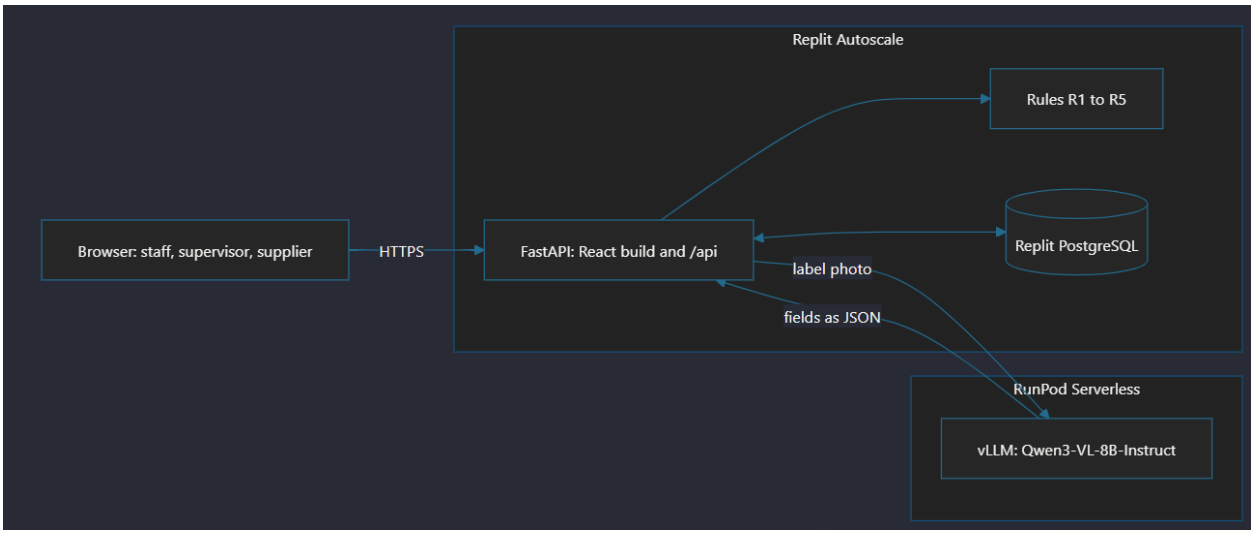
8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team’s knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



24/10/2026

Appendix A


Replit Autoscale
Rules R1 to R5
Browser: staff, supervisor, supplier
HTTPS
FastAPI: React build and /api
Replit PostgreSQL
label photo
fields as JSON
RunPod Serverless
vLLM: Qwen3-VL-8B-Instruct